NeoVerse Labs:

AI City Survival System

User Guide/README

C++ | 2026

Tables of Contents

Section	Topic	Page
1	System Overview	3
2	System File	4
3	How to Run	5
4	Default Login	5
5	Containers Used & Big-O Justification	6
6	Design Decision & Challenges Feed	8
7	Summary	9
8	Reference	10

1. System Overview

NeoVerse is a console-based C++ simulation of an AI managed smart city control system. It is built as a training prototype for the engineers who will eventually maintain real municipal AI infrastructure. The system models four core city subsystem (Power, Transport, Health and Security). It will respond autonomously to incoming events such as traffic accidents, power failures, network overloads and weather alerts. It will be monitored and manage the response by an authenticated engineer.

The program demonstrates dynamic data structures, algorithmic efficiency (with explicit Big-O analysis), object-oriented design and file-based. It runs simulation of a full engineer session like login, city monitoring, event processing, emergency resolution, reporting and data storage. This will be backed by STL containers and algorithms.

Key Features

- Engineer Authentication (1.1) – Engineers log in via username and password. With the passwords it will be stored as hashes (never plaintext). Both linear and binary search login methods are implemented for direct performance comparison. Clearance levels (Low/Medium/High) determine what actions an engineer may perform.
- City Data Storage (2) – Sensor readings are stored in a “vector” for fast indexed access. Historical city logs are stored in a “list” for efficient unbounded growth.
- Event Management (3) – Incoming city events are processed in arrival order via a “queue” (FIFO). Emergency overrides are resolved most-recent-first via a “stack” (LIFO). Events are further classified as critical or non-critical using STL algorithms.
- Polymorphic City Systems (4) – “PowerSystem”, “TransportSystem”, “HealthSystem” and “SecuritySystem” all inherit from an abstract “CityComponent” base class. Each overriding “processEvent()” to respond differently to the same event broadcast. Which is a direct demonstration of polymorphism.
- STL Algorithm Usage (5) – “sort”, “find”, “find_if”, “min_element”, “max_element" and “count_if” are used throughout. To the sort sensor data, search event/log records and identify extremes and critical thresholds.
- System Reporting (6) – Generates live reports. Like your total event processed, most common emergency type, average response time and a full system load summary. With safe handling of edge cases example like zero events processed.
- File Persistence (7) – Engineer accounts runs across via the “engineers.dat”. With the event and city log data are saved each sessions to “events.dat” and “city_logs.dat”. A “config.txt” file allows the critical-alert threshold to be configured externally without recompiling. A CSV export “event_report.csv” supports external analysis e.g. in excel.

2. System File

File	Purpose
main.cpp	Program entry point; Orchestrates login event simulation, reporting and file I/O
Engineer.h / Engineer.cpp	Engineer class: credentials, hashed password, clearance level
CityComponent.h / CityComponent.cpp	Abstract base class for all city systems, Defines the polymorphic interface
PowerSystem.h / .cpp	Manages power grid; handles power failure events
TransportSystem.h / cpp	Manages traffics flow; handles traffic accident events
HealthSysten.h / .cpp	Manages hospital capacity; handles weather alert events
Security.h / .cpp	Manages threat monitoring; handles network overload events
SensorReading.h	Struct representing one city sensor data record
engineers.dat	Persisted engineer accounts (auto-created on first run)
events.dat	Saved log of processed events and response times
city_logs.dat	Saved historical city log entries
config.txt	Configurable critical-alert threshold
event_report.csv	CSV export of the event log for external analysis

3. How to Run

Requirements: g++ (MinGW-w64/UCRT64 or something similar), C++11 or later.

- Open a terminal in the project folder that contains all .cpp/.h files.
- Compile all source files together using the wildcard. Do not compile individual files separately. As the project is split across multiple .cpp files and a single-file compile will fail with linker errors: "**g++ *.cpp -o neoverse.exe**"
- After words to run the program: "**.\neoverse.exe**"; must be entered.\
- On first run, the program will automatically create "*engineers.dat*" and "*config.txt*". With default sample data if they do not already exist. On subsequent runs, it loads the existing saved data.

4. Default Login Credentials

Username	Password	Clearance Level	Permission
Jane	pass123	High	Can process events and resolve emergency overrides
John	word456	Medium	Can process evets only (cannot resolve emergencies)
Sam	test789	Low	View-only; cannot process events or resolve emergencies

Note: Usernames and passwords are case-sensitive.

5. Containers Used and Big-O Justification

Engineer Storage & Login Search

Engineers are stored in a “vector<Engineer>” for contiguous, indexable storage.

- Linear search (loginLinear): O(n) checks each engineer sequentially. No ordering requirement. Suitable since the engineer list is small. The changes are infrequent relative to the size of the dataset in this prototype.
- Binary search (loginBinary): O(log n) search but does require the vector to first be sorted. O(n log n) for the sort itself. To perform on a copy to avoid disturbing the original vector’s order. Binary search is only worthwhile when searches vastly outnumber insertions. For a small engineer roster, the performance difference is negligible in practice. Although the comparison shows the trade-off clearly.

City Data Storage

- Vector<SensorReading> - chosen for sensor readings because it offers O(1) random access and cache friendly traversal. That is needed for the sorting/searching operations performed later in Question 5. Insertion at the end (push_back) is O(1). The removal from the front **(**erase(begin())**)** is O(n), since all subsequent elements must shift.
- List<string> - chosen for city logs because logs grow unbounded and are rarely accessed by index. Only when appended to traversed sequentially. Insertion/removal at the front (pop_front) is O(n) for both containers/ Although list traversal has weaker cache locality due to non-contiguous memory.

Event Handling

- Queue<string> - FIFO structure for incoming events, ensuring events are processed. Only in arrival order, matching real-world expectations for routine event handling. “push/pop/front” are all O(1).
- Stack<string> - LIFO structure for emergency overrides, ensuring the most recent emergency is resolved first. “push/pop/front” are all O(1).

STL Algorithms (Section 5)

Algorithm	Complexity	Justification
sort()	O(n log n)	Orders sensor reading by energy usage for reporting and analysis
find_if()	O(n)	Searches for a specific event/reading by a custom condition (exact find() only supports direct value equality, insufficient for struct/pair searches)
min_element() / max_element()	O(n) each	Single linear pass to locate extremes in energy usage
count_if()	O(n)	Counts critical readings/events matching a condition

File I/O (Section 7)

All the save/load operations are O(n). Looking at the number of records written or read. engineers.dat persists across runs with loaded at startup if present or created with sample data. events.dat and city_logs.dat are freshly generated and saved each sessions, reflecting that day's simulation rather than accumulated history. config.txt allows the critical alerts threshold to be changed externally without recompiling. Read once at the startup (O(n) in the number of configurations lines).

6. Design Decisions & Challenges Faced

Clearance based permissions: Rather than treating clearance level purely like a makeup attribute, it was tied to real functional restrictions. Low clearance can view system status but not act. Medium can process routine events. High clearance can resolve emergency overrides and everything else. This shows real-world access control, where the highest-stakes actions are restricted to the most of the trusted personnel.

Broadcast vs direct routing for events: When an event is dequeened or popped. It is broadcast to all four CityComponent objects rather than routed to a single matching component via a lookup table. Each subclass processEvent() already safely ignores irrelevant while still producing correct component specific response. I found a this topic and based my answer on (Team, 2025) and (apnatvar, 2026).

Why stack vs queue: Routine events like traffic accidents are processed in the order the occur. Since there is no inherent urgency ranking between them. A queue (FIFO) reflects this. Emergency overrides by contrast, representing situations serve enough to interrupt normal operation. The most recently declared emergency is typically the most time-sensitive. A stack (LIFO) handles this. I researched this one and applied into my task from (Team, 2025) and (Singh, 2024).

Memory lifetime bugs encountered and fixed: Two related bugs surfaced during development. Both stemming from C++ manual memory/object lifetime rules. The loginBinary initially returned a pointer into a temporarily sorted copy of the engineer vector. Which was destroyed when the function returned causing a dangling pointer. The CityComponents cleanup (delete) was initially placed before code that still needed to use objects, causing a use-after-free-crash. After all sections finished using the components pointer it was fixed by moving cleanup to the very end of the main().

Password hashing: Passwords are hashed once at account creation using “std::hash<string>” and only the has is stored. Login compares the hash of the entered password against the stored hash.

7. Summary

NeoVerse shows a complete, working simulation of an AI-managed city control system. Bulit on entirely STL containers and algorithms chosen deliberately for their complexity characteristics. An object-oriented class hierarchy shows the polymorphism and persistence of file-based state. Beyond satisfying individual rubric requirements, the system integrates them into one coherent simulation. Engineer authentication gates access to event handling, which mutates real component state. It feeds into reporting persistence by reflecting how these concepts functions together. In an actual smart-city control system rather than as isolated exercises.

References

apnatvar, 2026. *GitHub*. [Online]
Available at: https://github.com/apnatvar/software-architect-super-decision-making-skill/blob/main/skills/realtime/event-broadcast-vs-direct-messaging/event-broadcast-vs-direct-messaging-skill.md
[Accessed 19 September 2026].

Singh, R., 2024. *Medium*. [Online]
Available at: https://medium.com/@RobuRishabh/understanding-how-to-use-stack-queues-c-9f1fc06d1c5e
[Accessed 19 September 2026].

Team, G., 2025. *GeeksForGeeks*. [Online]
Available at: https://www.geeksforgeeks.org/computer-networks/broadcast-routing/
[Accessed 19 September 2026].

Team, G., 2025. *GeeksForGeeks*. [Online]
Available at: https://www.geeksforgeeks.org/cpp/stack-and-queue-c-cpp-programs/
[Accessed 19 September 2026].